

CS 316: Introduction to Deep Learning

Numerical Stability and Initialisation Week 7

Dr Abdul Samad

Lecture Outline

- Vanishing and Exploding Gradients
- Breaking Symmetry
- Xavier initialization

Vanishing and Exploding Gradients

- Consider a deep network with L layers, input $\mathbf{x}$, and output $\mathbf{o}$.
- With each layer l being defined by a transformation f_l parameterized by weights $\mathbf{W}^{(l)}$ and whose hidden layer output is $\mathbf{h}^{(l)}$, our network can be expressed as:

$$\mathbf{h}^{(l)} = f_l(\mathbf{h}^{(l-1)}) \text{ and thus } \mathbf{o} = f_L \circ \dots \circ f_1(\mathbf{x}).$$

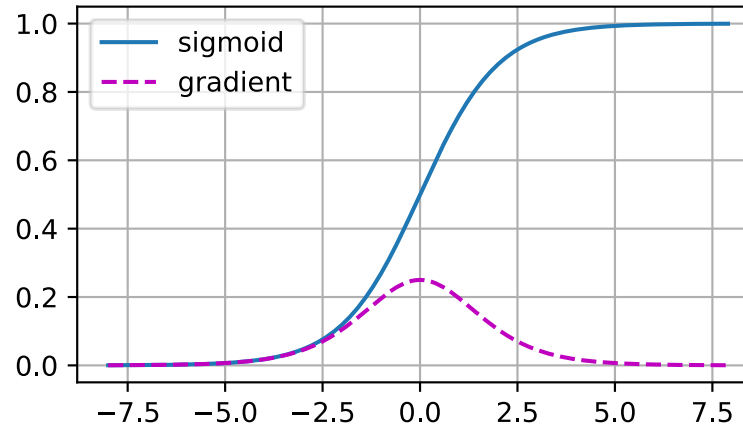
- We can write the gradient of $\mathbf{o}$ with respect to any set of parameters $\mathbf{W}^{(l)}$ as follows:

$$\partial_{\mathbf{W}^{(l)}} \mathbf{o} = \underbrace{\partial_{\mathbf{h}^{(L-1)}} \mathbf{h}^{(L)}}_{\mathbf{M}^{(L)} \stackrel{\text{def}}{=}} \cdot \dots \cdot \underbrace{\partial_{\mathbf{h}^{(l)}} \mathbf{h}^{(l+1)}}_{\mathbf{M}^{(l+1)} \stackrel{\text{def}}{=}} \underbrace{\partial_{\mathbf{W}^{(l)}} \mathbf{h}^{(l)}}_{\mathbf{v}^{(l)} \stackrel{\text{def}}{=}}.$$

- This gradient is the product of $L - l$ matrices $\mathbf{M}^{(L)} \dots \mathbf{M}^{(l)}$ and the gradient vector $\mathbf{v}^{(l)}$.
- Consequently, we are susceptible to the problem of numerical underflow or overflow, leading to parameter updates that are excessively large or excessively small, rendering the learning impossible as the parameters hardly move after each update.

Vanishing Gradients

- The choice of the activation function that is applied following each layer's linear operations is a common cause of the vanishing gradients.



- In the graph above, we can see that the sigmoid's gradient vanishes both when its inputs are large and when they are small.
- Moreover, when backpropagating through many layers, unless the inputs to many of the sigmoid's are close to zero, the overall product may vanish. So unless we are careful, the gradient will likely be cut off at some layer.

Exploding Gradients

- The technique with which weights are initialized is a common cause of the exploding gradients.
- For example, sample 100 Gaussian Random matrices and then multiply them with some initial matrix M. Given variance $\sigma^2 = 1$, the matrix product explodes.

$$M = \begin{bmatrix} -0.1882 & 1.4522 & 0.1471 & -1.4465 \\ 0.9311 & -0.4740 & -0.2528 & -0.7200 \\ -1.1675 & -1.2031 & 0.0862 & -0.9560 \\ 0.6061 & 0.7875 & -0.1915 & -0.8064 \end{bmatrix}$$

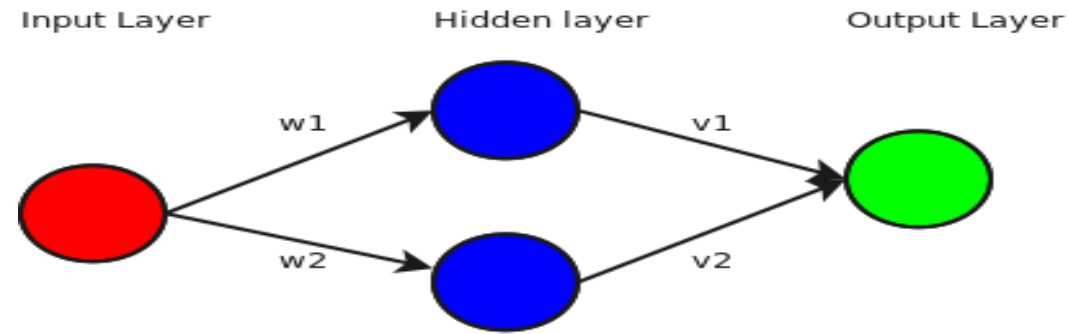
- After multiplying with 100 Gaussian random matrices, the matrix product is as follows:

$$\text{Result} = \begin{bmatrix} 6.9766e+23 & 3.6144e+24 & -1.8983e+24 & 2.5633e+24 \\ -2.0041e+22 & -1.0383e+23 & 5.4530e+22 & -7.3634e+22 \\ -5.2698e+22 & -2.7301e+23 & 1.4339e+23 & -1.9362e+23 \\ 3.8219e+23 & 1.9800e+24 & -1.0399e+24 & 1.4042e+24 \end{bmatrix},$$

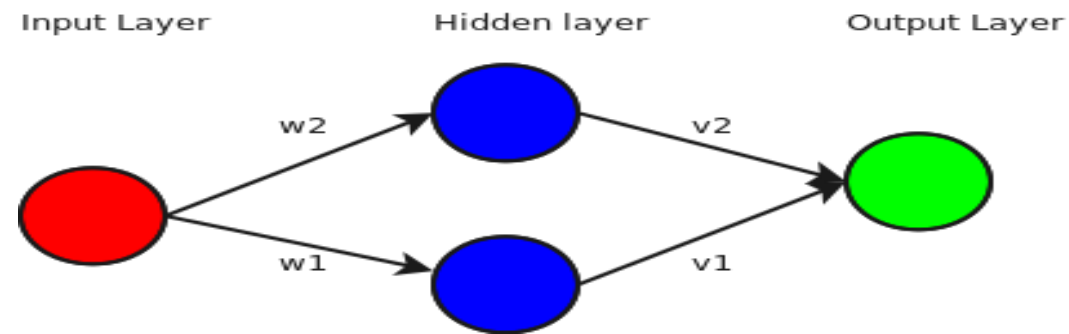
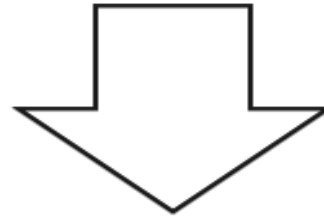
Breaking Symmetry

- Another problem in neural network design is the inherent symmetry in their parameterization.
- For example, consider the MLP with one hidden layer containing two hidden units.
- In this case, we could permute the weights of the first layer and likewise permute the weights of the output layer to obtain the same function. In other words, we have permutation symmetry among the hidden units of each layer.
- In addition, if we initialized all the parameters of the hidden layer with a constant value. During the forward propagation, each hidden unit would have the same value and during backward propagation, each hidden unit would have the same gradient. As a result, each unit will have the same value and the layer would behave as if it only had a single unit.

Breaking Symmetry



Swapping $w1$ and $w2$, and swapping $v1$ and $v2$ is a symmetry because the output is unchanged



Xavier Initialization

- With n_{in} inputs x_j and their associated weights w_{ij} for this layer, an output is given by:

$$o_i = \sum_{j=1}^{n_{in}} w_{ij} x_j$$

- The weights w_{ij} are all drawn independently from the same distribution. Furthermore, assume the distribution has zero mean and variance σ^2 .
- The inputs x_j are all drawn independently from the same distribution. Furthermore, this distribution has zero mean and variance γ^2 .

Xavier Initialization

- We can calculate the mean of output o_i as follows:

$$E[o_i] = \sum_{j=1}^{n_{in}} E[w_{ij}x_j] = \sum_{j=1}^{n_{in}} E[w_{ij}]E[x_j]$$

$$E[o_i] = \sum_{j=1}^{n_{in}} E[w_{ij}]E[x_j]$$

$$E[o_i] = \sum_{j=1}^{n_{in}} E[w_{ij}]E[x_j]$$

Xavier Initialization

- We can calculate the variance of output o_i as follows:

$$\text{Var}[o_i] = E[o_i^2] - (E[o_i])^2$$

$$\text{Var}[o_i] = \sum_{j=1}^{n_{in}} E[w_{ij}^2 x_j^2] - 0$$

$$\text{Var}[o_i] = \sum_{j=1}^{n_{in}} E[w_{ij}^2] E[x_j^2]$$

$$\text{Var}[o_i] = n_{in} \sigma^2 \gamma^2$$

Xavier Initialization

- One way to keep the variance fixed is to set $n_{in}\sigma^2 = 1$.
- Using the same reasoning as for forward propagations, we see that the gradient's variance can blow up unless $n_{out}\sigma^2 = 1$ where n_{out} is the number of outputs of this layers.
- We cannot possibly satisfy both conditions simultaneously. Consequently, we try to satisfy:

$$\frac{1}{2}(n_{in} + n_{out})\sigma^2 = 1 \text{ or equivalently } \sigma = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

- Typically, the Xavier initialization samples weights from a Gaussian distribution with zero mean and variance.
- We can also adapt Xavier's intuition to choose the variance when sampling weights from a uniform distribution.

$$U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

References

- https://d2l.ai/chapter_multilayer-perceptrons/numerical-stability-and-init.html

CS 316: Introduction to Deep Learning

Generalization in Deep Learning Week 8

Dr Abdul Samad

Lecture Outline

- Generalized Models
- Training and Generalized Error
- Model Complexity
- Model Selection
- Validation Dataset
- Overfitting vs Underfitting

Generalized Models

- As machine learning scientists, our goal is to discover patterns.
- But how can we be sure that we have truly discovered a general pattern and not simply memorized our data?
- When we deploy the model in the future, we will encounter examples that the model has never seen before.
- Our predictions will only be useful if our model has truly discovered a general pattern.
- Our goal is to discover patterns that capture regularities in the underlying population from which our training set was drawn.

Training and Generalization Error

- The training error is the error of our model as calculated on the training dataset.
- Generalization error is the expectation of our model's error were we to apply it to an infinite stream of additional data examples drawn from the same underlying data distribution as our original sample.
- We can never calculate the generalization error exactly.
- In practice, we must estimate the generalization error by applying our model to an independent test set constituted of a random selection of data examples that were withheld from our training set.

Model Complexity

- When we have simple models and abundant data, we expect the generalization error to resemble the training error.
- When we work with more complex models and fewer examples, we expect the training error to go down but the generalization gap to grow.

Factors affecting generalizability of a model

- Some of the factors that tend to influence the generalizability of a model class are as follows:
 - The number of tunable parameters. When the number of tunable parameters is large, models tend to be more susceptible to overfitting.
 - The values taken by the parameters. When weights take a wider range of values, models can be more susceptible to overfitting.
 - The number of training examples. It is trivially easy to overfit a dataset containing only one or two examples even if your model is simple. But overfitting a dataset with millions of examples requires an extremely flexible model.

Model Selection

- In machine learning, we usually select out final model after evaluating several candidate models. This process is called model selection.
- Sometimes models subject to comparison are fundamentally different in nature.
- At other times, we are comparing members of the same class of models that have been trained with different hyperparameter settings.
- In order to determine, the best among our candidate models, we will typically employ a validation dataset.

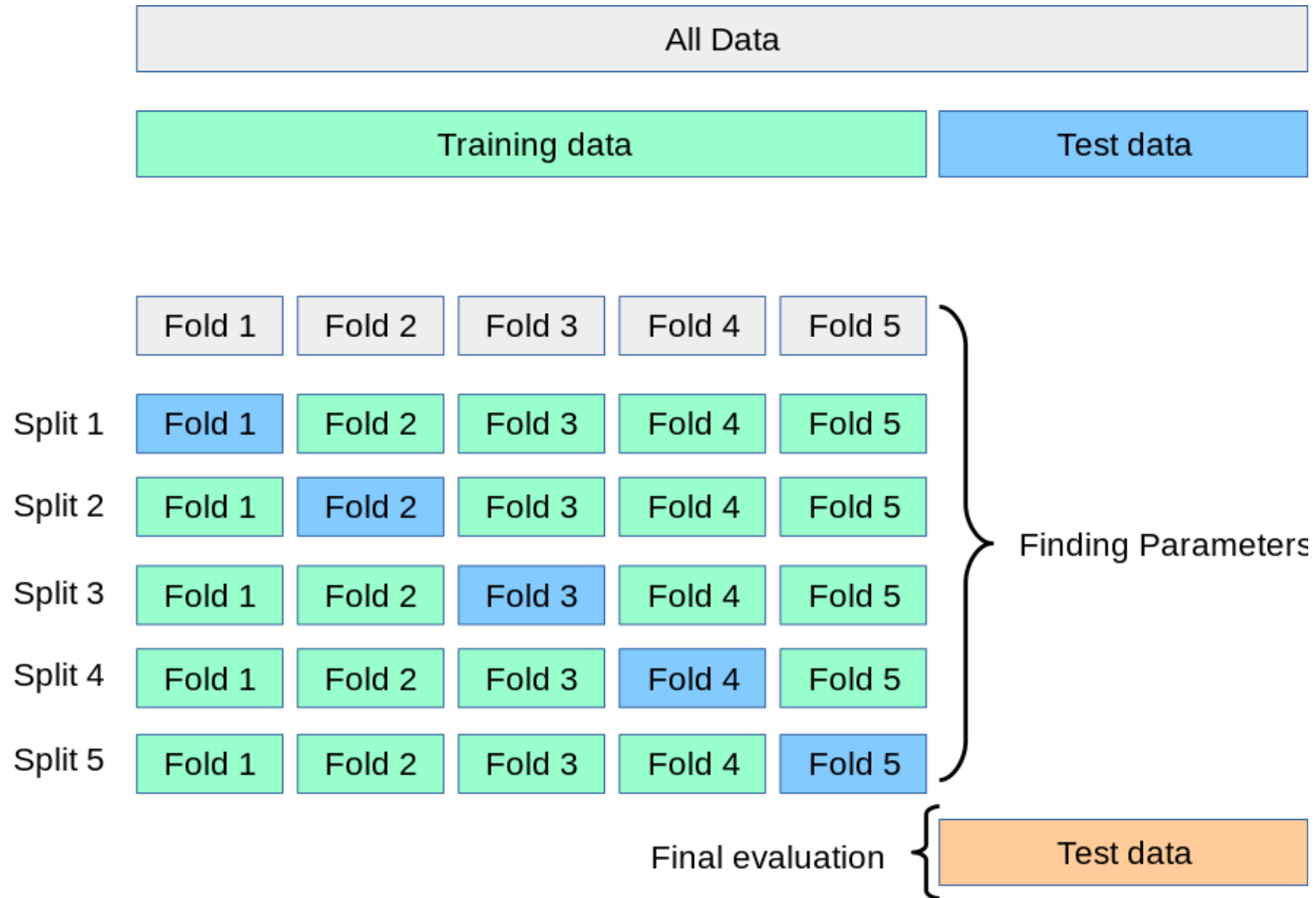
Validation Dataset

- In principle we should not touch our test set until after we have chosen all our hyperparameters.
- If we overfit our training data, there is always the evaluation on the test data.
- But if we overfit the dataset, how would we ever know ?
- We should never rely on the test data for model selection. And yet we cannot solely rely on the training data for model selection either because we cannot estimate the generalization error on the very data, we use to train the model.
- In the real world, test data is discarded after just one use, and we can seldom afford a new test for each round of experiments.
- The common practice to address this problem is to split our data three ways, incorporating a validation dataset into addition to the training and test datasets.

K-Fold Cross-Validation

- When the training data is scarce, we might not even be able to afford to hold out enough data to constitute a proper validation set.
- One popular solution to this problem is to employ K-fold cross validation.
- The original training data is split into K non-overlapping subsets.
- Model training and validation are executed K times, each time training on K - 1 subsets and validation on a different subset i.e., the one not used for training in that round.
- The training and validation errors are estimated by averaging over the results from the K experiments.

K-Fold Cross-Validation



Overfitting and Underfitting

- When comparing training and validation errors, we must keep two common scenarios in mind: **underfitting** and **overfitting**.
- If the model cannot reduce training error, this may indicate that our model is too simple to capture the pattern we are attempting to model. Furthermore, because the difference in generalization between our training and validation errors is small, we could employ a more complex model. This is referred to as **underfitting**.
- On the other hand, **overfitting** occurs when our training error is significantly lower than our validation error.
- Whether we overfit or underfit can depend both on the **complexity of our model** and the **size of the available training datasets**.

Influence of model complexity on underfitting and overfitting

- A complex model has more parameters than a less complex model and the model function's selection range is wider.
- Fixing the training dataset, complex models should always lower (at worse, equal) training error relative to less complex models.
- The relationship between model complexity and underfitting vs. overfitting can be visualized as follows.

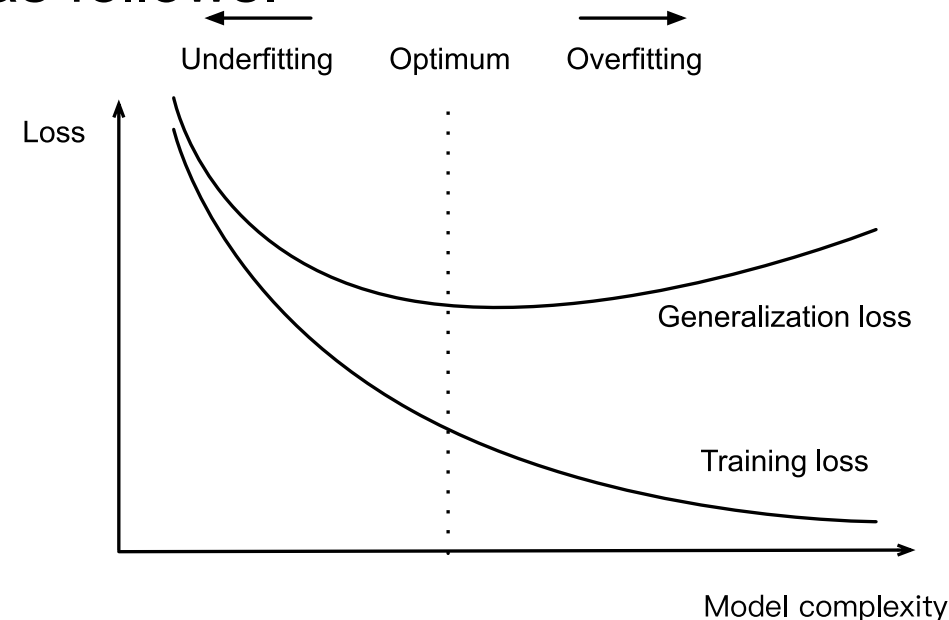


Figure taken from [Capacity vs Error](#)

Influence of model complexity on underfitting and overfitting

- Fixing our model, the fewer samples we have in the training dataset, the more likely we are to encounter overfitting.
- As we increase the amount of training data, the generalization error typically decreases.
- Given more data, we might profitably attempt to fit a more complex model.
- In the absence of sufficient data, simpler models may become more difficult to beat.

References

- https://d2l.ai/chapter_multilayer-perceptrons/generalization-deep.html

CS 316: Introduction to Deep Learning

Regularization Techniques Week 1

Dr Abdul Samad

Outline

- Weight Decay
- Regularized Loss
- L2 Norm vs L1 Norm
- Early Stopping
- Dropout

Weight Decay

- Weight decay is the one of most widely used technique for regularizing parametric machine learning models.
- Instead of directly manipulating the number of parameters, weight decay, operates by restricting the values that the parameters can take.
- The technique is motivated by the basic intuition that among all functions f , the function $f = 0$ (assigning the value 0 to all the inputs) is in some sense the simplest, and that we can measure the complexity of a function by the distance of its parameters from zero.
- One simple interpretation might be to measure the complexity of a linear function $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ by some norm of its weight vector e.g. $\|\mathbf{w}\|_2$

Regularized Loss

- The most common method for ensuring a small weight vector is to add its norm as a penalty term to the problem of minimizing the loss. Thus, our new objective is minimizing the sum of prediction loss and penalty term.
- Our current loss function is currently as follows:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\mathbf{w}^T \mathbf{x}^{(i)} + b - y^{(i)})^2$$

- To penalize the size of the weight vector, we must somehow add its norm to the loss function while also allowing the model to trade off the standard loss for this new penalty.
- In practice, we characterize this tradeoff via the regularization constant λ , a non-negative hyperparameter that we fit using validation data:

$$L(w, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2$$

Updating Weights

- The minibatch stochastic gradient descent update for L2 norm (l_2) regularized regression is as follows:

$$\mathbf{w} = (1 - \eta\lambda)\mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + b - y^{(i)})$$

- As before, we update $\mathbf{w}$ based on the amount by which our estimate differs from our observation. However, we also shrink the size of $\mathbf{w}$ towards zero.
- Given the penalty term alone, our optimization algorithm decays the weight at each step of training
- In contrast to feature selection, weight decay offers us a continuous mechanism for adjusting the complexity of a function.
- Smaller values of λ correspond to less constrained $\mathbf{w}$, whereas larger values of λ constrain $\mathbf{w}$ more considerably.

L2 Norm vs L1 Norm

- L2 (l_2) norm places an outsize penalty on large components of the weight vector.
- This biases our learning algorithm towards models that distributes weights evenly across a large number of features.
- L1 (l_1) norm penalties lead to models that concentrates weights on a small set of features by clearing other weights to zero.
- This gives us an effective method for feature selection.

L2 Norm vs L1 Norm - Example

- $\mathbf{x} = [1,1,1,1]^T$
- $\mathbf{w}_1 = [0.25,0.25,0.25,0.25]^T$
- $\mathbf{w}_2 = [1,0,0,0]^T$
- $\mathbf{w}_1^T \mathbf{x} = \mathbf{w}_2^T \mathbf{x}$

Early stopping

- Early stopping is a classical technique for regularizing deep neural networks.
- Rather than directly constraining the values of the weights, one constrains the number of epochs of training.
- The most common way to determine the stopping criteria is to monitor validation error throughout the training and to cut off training when the validation error has not decreased by more than some small amount of ϵ for some number of epochs.
- Besides the potential to lead to better generalization, in the setting of noisy labels, another benefit of early stopping is the time saved.

Dropout

- Classical generalization theory states that to close the gap between train and test performance, we should for a simple model.
- One useful notion of simplicity is smoothness i.e., the function should not be sensitive to small changes to its input. For instance, when we classify images, we should expect that adding some random noise to the pixels should be mostly harmless.
- Dropout involves injecting noise while computing each internal layer during forward propagation.
- The method is called dropout because we literally drop out some neurons during training. Throughout training, on each iteration, standard dropout consists of zeroing out some fraction of the nodes in each layer before calculating the subsequent layer.

Dropout

- One idea is to inject the noise in an unbiased manner such that the expected value of each layer – while fixing the others – equals to the value it would have taken absent the noise.
- At each iteration, noise sampled from a distribution with zero mean $\epsilon \sim N(0, \sigma^2)$ is added to the input $\mathbf{x}$.

$$\begin{aligned}\mathbf{x}' &= \mathbf{x} + \epsilon \\ E[\mathbf{x}'] &= E[\mathbf{x}]\end{aligned}$$

Standard Dropout Regularization

- In standard dropout regularization, one zeros out some fraction of the nodes in each layer and then debiases each layer by normalizing by the fraction of nodes that were retained.
- With dropout probability p , each intermediate activation h is replaced by a random variable h' :

$$h' = \begin{cases} 0 & \text{with probability } p \\ \frac{h}{1-p} & \text{otherwise} \end{cases}$$
$$E[h'] = E[h]$$

Dropout in Practice

- When we apply drop out to a hidden layer, zeroing out each hidden unit with the probability p , the result can be viewed as a network containing only a subset of the original neurons.
- Consider a MLP with a single hidden layer and 5 hidden units ($h_1, \dots, h_5$). After dropout, h_2 and h_5 are removed.
- The calculation of the outputs no longer depends on h_2 or h_5 and their respective gradient also vanishes when performing backpropagation.

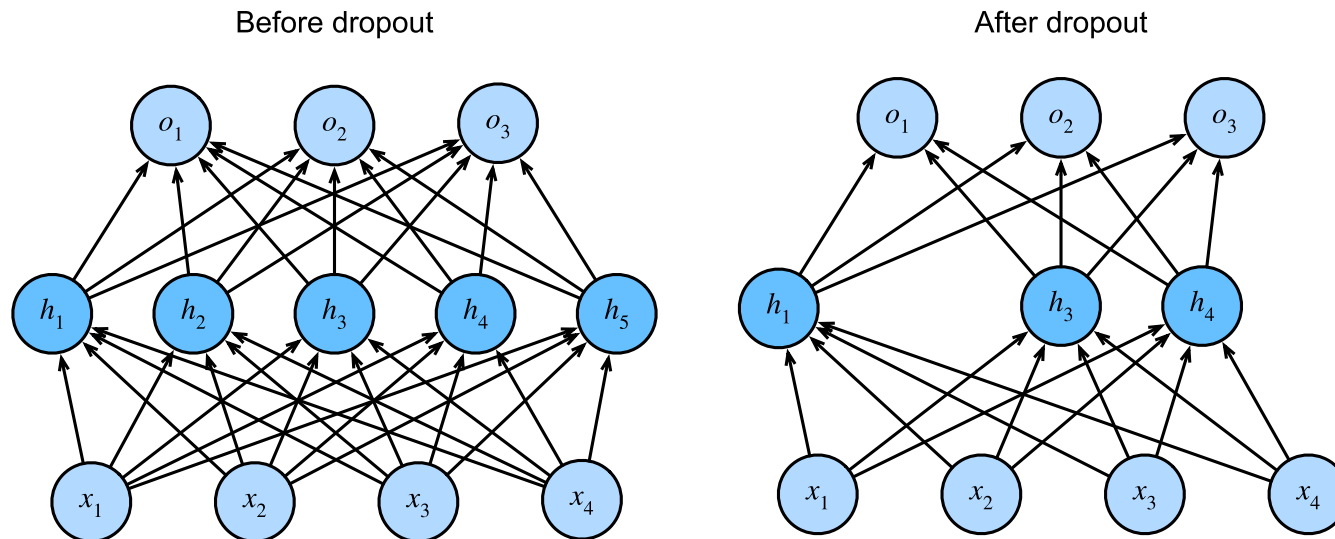


Figure taken from [Dropout](#)

References

- https://d2l.ai/chapter_linear-regression/weight-decay.html
- https://d2l.ai/chapter_multilayer-perceptrons/generalization-deep.html
- https://d2l.ai/chapter_multilayer-perceptrons/dropout.html